

Stat 442 Lecture Notes

Spring Semester, 2026

Prof. Anna Neufeld

Last Updated: February 9, 2026

1 Wednesday 2/4: Welcome, what is statistical learning, basics of supervised learning

1.1 What is statistical learning?

Statistical learning very broadly refers to a set of tools for *learning* something from data.

Consider the following three scenarios:

- **Scenario 1:** A researcher comes up with the hypothesis that college students who sleep more than 7 hours per night have higher GPAs, on average, than those who sleep fewer than 7 hours per night. She goes out and collects a random sample of students, and asks each of them (1) if they sleep more than 7 hours per night, and (2) for their GPA.

She then fits the linear model

$$\widehat{GPA} = \hat{\beta}_0 + \hat{\beta}_{sleep}(\text{sleep_more_than_7}),$$

and uses the estimated value $\hat{\beta}_{sleep}$, along with its associated p-value and confidence interval in R, to answer her research question.

- **Scenario 2:** A researcher is interested in what factors impact the GPA of a college student, so she collects a random sample of students and writes down a lot of information about them. She searches through the data, tries different models, looks at plots, and ends up realizing that hours of sleep per night is an important factor in determining GPA. As the relationship between hours of sleep per night and GPA does not look linear, she makes a new binary variable: yes/no do you sleep more than 7 hours per night.

At the end of this process, she fits the linear model

$$\widehat{GPA} = \hat{\beta}_0 + \hat{\beta}_{sleep}(\text{sleep_more_than_7}).$$

- **Scenario 3:** A researcher is interested in predicting the GPA of a college student based on various factors. She collects a random sample of students and writes down a lot of information about them. Without bothering to look at plots, she asks R to fit her a very complicated model that takes in all possible input variables $X_1, \dots, X_p$ and allows them to contribute non-linearly and in interacting ways to produce an estimate $\widehat{GPA}$. She sees that, on her dataset, her predictions are quite accurate ($R^2 = 0.98$).

Based on our definition, all three of these scenarios are examples of statistical learning. In scenario 1, we *learned* the value of $\hat{\beta}_{sleep}$: this was an estimate of a pre-specified parameter β_{sleep} . In scenario 2, we *learned* not only $\hat{\beta}_{sleep}$, but we also learned to be interested in the parameter β_{sleep} via a process of *variable selection* (choosing “sleep”) and *feature engineering* (deciding to make it binary). Finally, in scenario 3, we *learned* a prediction function, which could be applied to new datapoints.

While all three of these scenarios are examples of statistical learning, only scenario 1 is an example of classical statistics. Classical statistics assumes that our data analysis plan is completely pre-specified before looking at our data; this is the only setting in which classical confidence intervals and p-values are formally valid. When we talk about statistical learning, we are often focusing on methods and strategies that move beyond classical statistics: we are often discovering patterns in complex data without a specific, pre-specified plan. Statistical learning is often much more exploratory than classical statistics!

There is nothing “right” or “wrong” about the approaches in scenarios 1-3; it all depends on the goals of the researcher. In scenario 1, we get to rigorously test a hypothesis using tools we love (p-values!), but we sacrifice the flexibility to explore our data and uncover new ideas. In scenario 2, we get to be a lot more flexible and adaptive, but we give up our rigorous hypothesis testing, and manually trying so many different models might get tedious. If our goal is truly prediction, rather than inference, perhaps we like scenario 3, which lets us build a maximally flexible model without too much manual effort.

With this very broad definition of statistical learning in mind, we can now get into some specifics.

1.2 Basic terminology, and types of learning

In statistical learning, we use n to denote the number of observations. For $i = 1, \dots, n$, we measure p *covariates* or *predictors*, $x_{i1}, x_{i2}, \dots, x_{ip}$. We denote the entire matrix of observed covariate with $\mathbf{X}$, which has dimensions $n \times p$. I will typically use capital, bolded letters to denote matrices.

In supervised learning, we also measure a response variable y_i for each observation. I will typically use bold lowercase letters to denote vectors of length n , so we could collect these observations all together to make $\mathbf{y}$.

Notationally, I will use capital, unbold letters to denote random variables, which could be either vectors or scalars. I use lowercase letters for observed values. In our example from Section 1.1, we let X_1 denote the number of hours that a randomly selected student sleeps per night and let Y denote their GPA. Once we observe a particular n students for our dataset, we let their sleep values be $\mathbf{x}_1 = (x_{11}, x_{12}, \dots, x_{1n})^\top$ and we let their GPA values be $\mathbf{y} = (y_1, \dots, y_n)^\top$.

Putting it all together, the data that we start with for a supervised learning problem is a matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ and a response vector $\mathbf{y} \in \mathbb{R}^n$, and we are generally interested in using the predictor variables to explain or predict the response variable. We usually hope that whatever we learn would generalize to hypothetical new realizations (X, Y) .

In contrast, the setting with no response variable y where we just want to uncover patterns in $\mathbf{X}$ is known as *unsupervised learning*.

In this course, we will spend around 9 weeks on supervised learning, and only 2 weeks at the end on unsupervised learning. However, I do not want to give you the impression that unsupervised learning is less important! Unsupervised learning is extremely important, and is in fact the back-bone of recent generative AI technology. We start with supervised learning because it has a more natural connection to classical statistics and builds more naturally on your previous work. Hopefully, some of the concepts that you learn during our supervised learning unit will help you be equipped to delve more deeply into unsupervised learning in the future if you choose.

There is one more type of learning that has become really important in recent years. This is the field of *semi-supervised learning*, in which we have a (potentially small) set of *labeled data* (data with predictors + response) and a potentially much larger set of *unlabeled data* (data that is missing the response). While we will not cover this in lecture, there are some great potential final project topics related to semi-supervised learning.

1.2.1 A side-bar about counting n and p

We didn't talk about this in class, but I want to quickly make the point that counting n and p is not always trivial. Consider a dataset with one categorical variable that takes on values $\{\text{cat}, \text{dog}, \text{hamster}, \text{rabbit}\}$. I could represent this variable using an $\mathbf{X}$ with one column that stores $\{\text{dog}, \text{dog}, \text{cat}, \text{dog}, \text{hamster}, \text{cat}, \text{dog}, \dots\}$. However, for the purpose of model fitting, it is likely going to be convenient to instead let $\mathbf{X}$ be a matrix with four columns, that store binary variables indicating "is this a dog", "is this a cat", etc. For the purposes of parameter-counting, this is now a dataset with $p = 4$, not $p = 1$.

Similarly, suppose we take measurements on 920 students, who are spread out between 8 schools. The schools are randomly assigned to either have required yoga PE class, or to have traditional PE class, and we measure resulting test scores at the end of the year. While we have 920 students, the students within a given school are not *independent* of one another. On the other hand, the schools underwent random assignment and can be safely assumed to be independent of one another. Thus, is the sample size n the 920 students or the 8 schools? It turns out that for this *clustered* or *hierarchical* data, we have a notion of *effective sample size* that is somewhere between 920 and 8, and depends on how correlated the students are within the schools.

We should also mention that data does not always come to you in a form where it looks like $\mathbf{X} \in \mathbb{R}^{n \times p}$ and $\mathbf{y} \in \mathbb{R}^n$. Consider the task of image classification. You get a set of 500 images, which are stored as 400×400 pixel images, where each pixel records a numerical value between 0 and 255 for red, green, and blue. Each image is associated with a label in $\{\text{cat}, \text{dog}\}$, and you observe $y_1, \dots, y_{500}$. In this case, the RGB values in the pixels in the images are your predictor values, but they don't come in a simple $n \times p$ matrix. We need to reshape the data so that each image i has an associated vector $x_i \in \mathbb{R}^{400 \times 400 \times 3}$ that stores all three color values for all 400×400 pixels. So p is very large! It will also likely be convenient to code y_i such that a 1 denotes a dog and a 0 denotes a cat, such that y becomes numerical. When there are more than 2 classes, such as $\{\text{cat}, \text{dog}, \text{fish}\}$, we will need more than one column to store our response variables numerically.

Who knew that counting n and p could be so complicated! In this class, just think of it as the number of rows and columns (minus the response columns) of the data, and remember these subtleties in case they ever come up.

1.3 Goals of supervised learning

Generally in supervised learning, we assume that there is some relationship between Y and X , such that we can write Y as

$$Y = f(X) + \epsilon.$$

The function $f()$ is a fixed but unknown function that represents the systematic information that X provides about Y , and ϵ is a random error term, which should satisfy the condition that $E[\epsilon] = 0$ and that ϵ is independent of X . These conditions ensure that $f()$ is capturing all parts of Y that can be explained by X , and ϵ captures the parts of Y that cannot be explained by X .

The main task of supervised learning is: given (x_i, y_i) for $i = 1, \dots, n$, can we estimate f ? Meaning, can we come up with a function $\hat{f}()$ such that, $Y \approx \hat{f}(X)$?

When Y is numerical, this task is called regression. When Y is categorical, this task is called classification.

There are two reasons why we might want to come up with a function $\hat{f}$.

1. **Prediction:** We might want to be able to come up with predictions $\hat{Y} = \hat{f}(X)$ for future realizations of data where Y itself is not observed.
2. **Interpretation:** We might want to understand which predictors in X are most associated with Y in order to draw a scientific conclusion, or at least take a step towards drawing a scientific conclusion.

As we will see throughout the semester, these two goals can sometimes be at odds with one another! If we only care about prediction, we do not need our function $\hat{f}()$ to be interpretable: we can use an extremely complex model (known as a “black box”), and we will be happy as long as $\hat{Y} \approx Y$. On the other hand, if we want to draw scientific conclusions, then we might wish to use a simple, interpretable model for $\hat{f}()$ that lends itself to rigorous inference. We should also note that it is not always an either-or situation. Sometimes, we want to make good predictions and also understand which variables are contributing significantly to the predictions. Managing the tradeoff, or lack thereof, between these two goals will be one of our fundamental themes for the semester. When starting a supervised learning problem, always think carefully about which of these goals (or both) is important to you!

1.4 What methods do you already know, and why will we move beyond them?

You all already know at least one method for regression (linear regression) and one method for classification (confusingly, named logistic regression). While these methods are taught in the most classical of statistics courses, they are certainly examples of machine learning methods!

Why will this class move beyond these methods?

Let's focus on the linear regression case, and suppose that we have p predictors. This means that we will let:

$$\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p$$

Over the next few weeks, we will focus on ways to make this model both *more* and *less* flexible. You have actually seen both of these concepts in Stat 346.

- We need ways to make this more flexible, because this simple linear form might not do a good job approximating Y . What if the relationship between X_1 and Y is quadratic, or is otherwise wiggly? What if we need interaction terms between variables?
- We also need ways to make this less flexible. You all know from Stat 346 that, if p is very large, we run into trouble. If $p > n$, there is no unique least squares solution to the linear regression problem. If $p < n$ but is otherwise large, fitting this linear regression is probably a bad idea: the model is so complex that we risk overfitting to our training set. If our predictors are redundant, the model can be very unstable.

You learned in Stat 346 that you can always add transformations of variables, or interaction terms, to make a model more flexible and still fit it in linear regression. You also learned that variable selection techniques or PCA can help reduce the complexity of a model to avoid overfitting or instability.

We will start by talking about these concepts (more and less flexible models) mainly in the context of things that at least look like linear regression. However, there is also no rule that says $\hat{f}()$ has to be a linear function!

In recent decades, as datasets have become larger and computers have become more powerful, there have been a proliferation of complex methods for estimating $\hat{f}()$. Some of these look like fancy linear regression, but some look entirely different! Why?

- Datasets have gotten bigger. As we will learn in this class, complex models are often only appropriate when applied to enough data. Thus, it is natural that the “big data” era has gone hand-in-hand with the development of complex statistical learning methods.

- Computers have gotten better, faster, and cheaper, which has enabled the fitting of complex models.
- Free, open-source software programs like R and python have allowed researchers to develop algorithms and then easily share them with non-experts. This has allowed many methods to become popular.

Given the huge number of statistical learning methods, we could spend each class this semester learning a new method, such that at the end you are left with a huge cookbook of methods to choose from for your future data analysis needs. In fact, this class will be a little bit like this. But, there is a problem with this approach of treating statistical learning like a cookbook. New machine learning methods get published every day, and the state-of-the-art is constantly changing. If your goal is to learn a list of methods or algorithms, you will find yourself continually behind and out-of-date! Thus, the goal of this class is not really to teach you a list of methods. Instead, the goal is to teach you the fundamental concepts and overarching themes that go into the development of the methods, such that you can compare methods, recognize their limitations, and learn new methods on your own in the future. This goal helps explain two features of this course:

- We will spend a fair amount of time on older, less state-of-the-art methods. We will use these methods to illustrate general principles and themes, even if they may not be the methods that you will use in practice in your future.
- You will all be doing an *algorithm presentation* this semester. You will be responsible for doing independent reading to learn about a method. You will need to synthesize what you learn, and give a 20 minute presentation to the class explaining the method. This skill mirrors the way you will all interact with machine learning in the future: you will be continually learning and synthesizing new methods.

2 Tuesday, Feb 10: The bias variance tradeoff, as illustrated with KNN and linear regression

2.1 Recap of last class

The fundamental task of regression is as follows.

We have realizations $x_1, \dots, x_n$ and $y_1, \dots, y_n$. The y_i are numerical scalars. The x_i can be vectors of length p .

We assume $Y = f(X) + \epsilon$, and we want to estimate f . Recall that our goal in estimating f could be either *prediction* or *interpretation*. For today, let's assume that our goal is mainly prediction: this is a more concrete and measurable goal.

We can never really expect to find f exactly without placing any assumptions or constraints on its functional form. Also, placing restrictions on the functional form might help us ensure that our model is *interpretable* (rather than a black box), if we also care about that!

Once we restrict our attention to $\mathcal{F}$, our goal in supervised learning is to find:

$$\hat{f}_{ideal} = \operatorname{argmin}_{f' \in \mathcal{F}} E_{X,Y} [\mathcal{L}(Y, f'(X))].$$

However, we don't have access to the joint distribution of X and Y . Thus, we rely on a training dataset, and we rely on the approximation

$$\hat{f} = \operatorname{argmin}_{f' \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f'(x_i)).$$

2.2 MSE and the "ideal" predictor

A very common choice of loss function is the mean squared error (MSE). Under this loss function,

$$\hat{f}_{ideal} = \operatorname{argmin}_{f' \in \mathcal{F}} E_{X,Y} [(y - f'(x))^2].$$

You all already know that, if we additionally let $\mathcal{F}$ be the set of all functions that can be written as $\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$, then this is just least squares linear regression.

For now, though, assume that we had absolutely no constraints on our function class $\mathcal{F}$. On Homework 1, you will argue that, in this case,

$$\hat{f}_{ideal}(x) = E[Y \mid X = x].$$

We will now focus on two very different ways to approximate $E[Y \mid X = x]$: linear regression and KNN. For simplicity, we will assume that $p = 1$ so that x is a scalar.

MSE has all sorts of beautiful mathematical properties as a loss function; one of which we will see today. However, it is far from the only choice of loss function for regression. Be sure that you can articulate some limitations and some advantages!

2.3 Regression with one predictor variable: linear regression vs. KNN case study

Let's review a really simple case. We have a single numerical response variable Y and a single numerical predictor variable X . We observe 100 datapoints, so $n = 100$ and $p = 1$. We assume that $Y = f(X) + \epsilon$, and we want to pick a $\hat{f}$ such that

Let's compare two methods for coming up with $\hat{f}$.

2.3.1 Simple linear regression

Linear regression makes the task of finding $\hat{f}$ easier by making a strong assumption that limits the complexity of the model class that we are considering.

In linear regression, we restrict our attention to $\hat{f}$ that have the form $b_0 + b_1x$ for $b_0, b_1 \in \mathbb{R}$. This reduces the problem of searching for

$$\hat{f} = \operatorname{argmin}_{f' \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n (y_i - f'(x_i))^2$$

to the problem of searching for

$$\hat{\beta}_0, \hat{\beta}_1 = \operatorname{argmin}_{b_0, b_1 \in \mathbb{R}} \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1x_i)^2,$$

and then we just let $\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1x$. A nice thing about limiting our model class in this way is that we are much better, in terms of math skills, at optimizing over numbers than we are at optimizing over functions. This now looks like a Math 150 problem!

Under the assumption that $E[Y] = \beta_0 + \beta_1X$ for some true, unknown β_0 and β_1 , this solution directly approximates the best possible function $E[Y | X]$. In fact, under this assumption, the least squares assumption is the BLUE: best linear unbiased estimator! Where best means least variance! However, if this assumption does *not* hold, then the linear model is too limiting. The result of limiting our model class so much is that we might have a lot of bias! We will talk about this more later.

You are all experts in linear regression! So we will not spend too much more space in the notes on it.

2.3.2 K-nearest neighbors

KNN is at the opposite end of the spectrum from linear regression. The premise is quite simple. KNN lets

$$\hat{f}(x) = \frac{1}{k} \sum_{x_i \in N_k(x)} y_i, \tag{1}$$

where $N_k(x)$ is a function that returns the k points in the training set that are closest to the input point x according to some distance metric (for us, Euclidean distance). So, avoiding equations: KNN makes predictions by finding the k training observations with x_i closest to x , and predicts that the response for x will be the average of the responses for those k points.

The hyper-parameter k is chosen by the user. When $k = n$, KNN just predicts $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ for all observations, regardless of x . As k decreases, the functions returned by KNN get increasingly wiggly and flexible. When $k = 1$, the KNN function is a step function that can be arbitrarily wiggly, and that always has 0 MSE on the training set.

Note that KNN directly approximates $E[Y | X = x]$ *locally*, making no assumptions on the structure of $E[Y | X]$. When k is small, the estimation of $E[Y | X = x]$ uses only data that are really close to x . This lets our prediction function get arbitrarily wiggly. However, with small k , there is a lot of variance in each individual prediction, since it is made using very few datapoints. When k is large, there is less variance in the predictions, but we do not let our predictions be as wiggly, and so we introduce more bias.

A few notes on KNN:

- KNN is non-parametric; we cannot write down the function $\hat{f}$ without storing basically the entire dataset. The number of effective parameters grows with the sample size n .

- Linear regression in theory takes some time to train, but it is nearly instantaneous to make new predictions. KNN involves no training step, but it could be computationally expensive to obtain new predictions.

2.4 Training error vs. test error

For a given problem, how could we tell if KNN with $k = 1$, KNN with $k = 3$, KNN with $k = 7$, or linear regression is a better choice of model?

We will see in this section that “training error” is a really bad way to check. We need test error.

2.4.1 Training error

If we use our dataset $\mathbf{y} \in \mathbb{R}^n$ and $\mathbf{x} \in \mathbb{R}^n$ to *train* a function $\hat{f}$, then our *training error* is:

$$MSE_{\text{train}} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2.$$

Unfortunately, training error is a really bad estimate of future model performance. The problem is that we used this same data (and, in fact, this same loss function) to choose our function $\hat{f}$. Thus, the more complex our function class $\mathcal{F}$ is, the lower this training error will be.

For a fixed function $\hat{f}$ that we already trained, if we observe some test points $x_i^{\text{test}}, y_i^{\text{test}}$ for $i = 1, \dots, n_{\text{test}}$, we can compute our test error as

$$MSE_{\text{test}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \left(y_i^{\text{test}} - \hat{f}(x_i^{\text{test}}) \right)^2,$$

where the important thing is that the points $x_i^{\text{test}}, y_i^{\text{test}}$ were not used to compute the function $\hat{f}$.

While in practice we usually do compute MSE_{test} over some fixed test set that has size n_{test} , we don’t want our notion of model performance to be specific to the test set that we happened to see. We are likely studying MSE_{test} to estimate how big our error will be on a random new datapoint. In this case, we might be computing this for the particular function $\hat{f}$ that we already computed. In this case, our expected prediction error conditional on our particular function $\hat{f}$ that we already computed is:

$$E_{X^{\text{test}}, Y^{\text{test}}} \left[\left(Y^{\text{test}} - \hat{f}(X^{\text{test}}) \right)^2 \right],$$

where the function $\hat{f}$ is not treated as random.

If we want to evaluate the potential of an algorithm or a procedure to make good predictions, then we want to take into account the randomness in our training set. We want to acknowledge that we could have seen a different training set that would have given us a different function $\hat{f}$. With this in mind, in the next section we will define a more complex notion of expected prediction error. This more complex notion will let us come up with a very beautiful decomposition!

2.5 The bias-variance tradeoff

The exact math of the bias-variance decomposition that we will discuss today holds only for squared error loss, but similar concepts apply for other loss functions.

Suppose that we train our model $\hat{f}$ on a dataset $\mathbf{X}^{\text{train}}, \mathbf{y}^{\text{train}}$. Since this dataset is random, the function $\hat{f}()$ is also random. Now, suppose that we would like to know about the expected prediction error for a new datapoint with $X = x^{\text{test}}$ and unknown $Y = y^{\text{test}}$. The prediction error itself is $\left(y^{\text{test}} - \hat{f}(x^{\text{test}}) \right)^2$, but to find the *expected* prediction error we need to take the expected value over multiple sources of randomness.

For our purposes, let’s treat x^{test} as fixed. We want to find the average prediction error that we would obtain if we repeatedly (1) sampled training sets of size n from the population, (2) refit the function $\hat{f}$ using this training data, and (3) evaluated the squared prediction error for a datapoint drawn with $X = x^{\text{test}}$. So, we want to evaluate:

$$E_{\mathbf{X}^{\text{train}}, \mathbf{y}^{\text{train}}, Y | X = x^{\text{test}}} \left[\left(Y - \hat{f}(X) \right)^2 \mid X = x^{\text{test}} \right] = E_{\mathbf{X}^{\text{train}}, \mathbf{y}^{\text{train}}, Y | X = x^{\text{test}}} \left[\left(Y - \hat{f}(x^{\text{test}}) \right)^2 \mid X = x^{\text{test}} \right].$$

For brevity, I am not going to keep writing the subscripts in the expected values. I am going to assume that we are treating x^{test} as fixed but taking the expected value over all other randomness. But remember that it is always good

to know what exactly you are taking the expected value over!

To break this quantity down, we are going to do a clever trick that involves adding and subtracting 0 twice. This is a very common proof technique! After we do this, we expand our big squared quantity by thinking of it as $(a + b + c)^2$, and we apply linearity of expectation to put each term in its own expected value.

$$\begin{aligned}
E \left[\left(Y - \hat{f}(x^{\text{test}}) \right)^2 \right] &= E \left[\left(Y - \textcolor{red}{f}(x^{\text{test}}) + \textcolor{red}{f}(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] + E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right)^2 \right] \\
&= E \left[\left(Y - f(x^{\text{test}}) \right)^2 \right] + E \left[\left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right)^2 \right] + E \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right)^2 \right] \\
&\quad + 2E \left[\left(Y - f(x^{\text{test}}) \right) \left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) \right] \\
&\quad + 2E \left[\left(Y - f(x^{\text{test}}) \right) \left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] \\
&\quad + 2E \left[\left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) \left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right].
\end{aligned}$$

We will now argue that each of the cross terms is 0. We start with the orange term, and note that:

$$\begin{aligned}
\textcolor{brown}{E} \left[\left(Y - f(x^{\text{test}}) \right) \left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) \right] &= \left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) E \left[Y - f(x^{\text{test}}) \right] \\
&= \left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) (E[Y] - f(x^{\text{test}})) \\
&= \left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) (f(x^{\text{test}}) - f(x^{\text{test}})) = 0.
\end{aligned}$$

The first two equalities follow because the quantities $f(x^{\text{test}})$ and $E[\hat{f}(x^{\text{test}})]$ are not random, since x^{test} is a constant. The last equality follows since, because we are conditioning on $X = x^{\text{test}}$, $Y = f(x^{\text{test}}) + \epsilon$, and so by our assumption that $E[\epsilon] = 0$, $E[Y] = f(x^{\text{test}})$.

We now consider the purple term. We note that $Y - f(x^{\text{test}})$ is independent of the training dataset, since $f()$ is non-random and Y is a new realization. On the other hand, $(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}))$ is a function only of the training data, since x^{test} is non-random. Thus, this term has the form $E[ab]$ where a and b are independent. So we can write it as:

$$\begin{aligned}
\textcolor{violet}{E} \left[\left(Y - f(x^{\text{test}}) \right) \left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] &= E \left[Y - f(x^{\text{test}}) \right] E \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] \\
&= E \left[Y - f(x^{\text{test}}) \right] \left(E[\hat{f}(x^{\text{test}})] - E[\hat{f}(x^{\text{test}})] \right) = 0.
\end{aligned}$$

Finally, we consider the green term.

$$\begin{aligned}
\textcolor{green}{E} \left[\left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) \left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] &= \left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) E \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] \\
&= \left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right) \left(E[\hat{f}(x^{\text{test}})] - E[\hat{f}(x^{\text{test}})] \right) = 0.
\end{aligned}$$

Now that we know that all three cross terms are 0, we know that

$$E \left[\left(Y - \hat{f}(x^{\text{test}}) \right)^2 \right] = E \left[\left(Y - f(x^{\text{test}}) \right)^2 \right] + E \left[\left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right)^2 \right] + E \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right)^2 \right].$$

Our next goal is to understand each of these terms!

1. The first term can be written as:

$$E \left[\left(Y - f(x^{\text{test}}) \right)^2 \right] = E \left[\left(f(x^{\text{test}}) + \epsilon - f(x^{\text{test}}) \right)^2 \right] = E \left[\epsilon^2 \right] = \text{Var}(\epsilon).$$

This is our irreducible error term! It comes from the inherent noise in our data that cannot be explained by X .

2. In the second term, $f(x^{\text{test}})$ and $E[\hat{f}(x^{\text{test}})]$ are not random, so we do not need the expected value! This just becomes

$$\left(f(x^{\text{test}}) - E[\hat{f}(x^{\text{test}})] \right)^2,$$

which we call the squared bias. Is $\hat{f}(x^{\text{test}})$ equal to $f(x^{\text{test}})$ on average? If so, then $\hat{f}$ is unbiased and this term will disappear.

3. Finally, in the third term:

$$E \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right)^2 \right] = \text{Var} \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] + E \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right]^2.$$

First, note that $\text{Var} \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] = \text{Var} \left[\hat{f}(x^{\text{test}}) \right]$, because $E[\hat{f}(x^{\text{test}})]$ is not random. Then, note that $E \left[\left(E[\hat{f}(x^{\text{test}})] - \hat{f}(x^{\text{test}}) \right) \right] = E[\hat{f}(x^{\text{test}})] - E[\hat{f}(x^{\text{test}})] = 0$. So, this third term is simply the variance (with respect to the training set) of $\hat{f}(x^{\text{test}})$.

This was a lot of work, but we can finally say that, at a given point x^{test} ,

$$E \left[\left(Y - \hat{f}(x^{\text{test}}) \right)^2 \right] = \text{Var}(\epsilon) + \text{Bias}(\hat{f}(x^{\text{test}}))^2 + \text{Var}(\hat{f}(x^{\text{test}})).$$

This is the bias variance decomposition, which will be important throughout the semester!

The $\text{Var}(\epsilon)$ term is our irreducible error. No matter how good we are at estimating f , this is just the amount of noise in our data, so we will always have this error. On the other hand, we can reduce the bias and the variance terms if we pick a better statistical learning algorithm for estimating f .

Without giving too much away: very simple models tend to have high bias but low variance. Very complex (wiggly) models have low bias (they are never constrained!) but very high variance (they overfit to the training data). To minimize our expected prediction error, we are always looking for functions that hit the sweet spot of complexity! This sweet-spot is usually at the minimum of a U-shaped curve for test-set error! You will make U-shaped curves on your homework this week!

2.6 Wrap-up

I am *sure* that we went to quickly through the proof. Next class, we will look at an RMarkdown document that will help reinforce the concept of the bias-variance tradeoff.

3 Thursday, Feb 13: adding more predictor variables!

Recall our typical regression setting. We assume that our data are i.i.d. realizations of random variables (X, Y) , where

$$Y = f(X) + \epsilon.$$

We assume that $E[\epsilon] = 0$ and $\epsilon \perp\!\!\!\perp X$, but the function $f()$ is unknown. Our goal is to find a function $\hat{f}$ such that $Y \approx \hat{f}(X)$.

More specifically, we would like to find $\hat{f}$ that minimizes the expected squared error loss for a new, unseen datapoint (X, Y) . So ideally, we are looking for

$$\operatorname{argmin}_{\hat{f} \in \mathcal{F}} E_{X,Y} \left[\left(Y - \hat{f}(X) \right)^2 \right], \quad (2)$$

where $\mathcal{F}$ is some class of functions. On your homework, you will argue that if $\mathcal{F}$ were totally unconstrained, we would want to set $\hat{f}(x) = E[Y | X = x]$. This argument was also in the Lecture 2 notes, but we skipped it.

This is where we hit practical issues. We do not know the joint distribution of X and Y , and so we cannot pick $\hat{f}$ to be this ideal function $E[Y | X = x]$. And we really cannot search efficiently over all possible real-valued functions $\mathcal{F}$ to find a good choice for $\hat{f}$. So, a statistical learning algorithm typically restricts the class $\mathcal{F}$ to make this task doable. So far in this class, we have discussed two possible methods for picking $\hat{f}$. It turns out that both of these can be viewed as approximating $E[Y | X = x]$; they just do this using different sets of assumptions.

Up until now, we have been assuming that we just have one predictor variable, and so X is a scalar. Today, we will let X be a vector, meaning that we have p predictor variables $X_1, \dots, X_p$. This is going to introduce a lot more nuance to the comparison between linear regression and KNN!

Agenda :

1. Linear regression in high dimensions.

2. KNN in high dimensions.
3. R demo, and overall comparison of linear regression vs. KNN, without preprocessing.

3.1 Linear regression in high dimensions

We restrict our model class $\mathcal{F}$ to

$$\mathcal{F} = \{f : f(x) = b_0 + b^T x, b_0 \in \mathbb{R}, b \in \mathbb{R}^p\},$$

such that (2) becomes

$$\operatorname{argmin}_{b_0 \in \mathbb{R}, b \in \mathbb{R}^p} E_{X,Y} \left[(Y - b_0 - b^T X)^2 \right]. \quad (3)$$

As you know from Stat 346, it is convenient to append a column of 1s to our predictor matrix $\mathbf{X}$ so that we are just optimizing over a single $b \in \mathbb{R}^{p+1}$. That way, we don't need to keep writing the intercept separately. Adopting this convention, with linear regression we approximate (3) by letting

$$\hat{\beta} = \arg \min_{b \in \mathbb{R}^{p+1}} \sum_{i=1}^n (y_i - b^T x_i)^2. \quad (4)$$

You know 1,000 things about this estimator from Stat 346. For example, as long as $n > p$, the solution to (4) has a closed-form: $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y$. Furthermore, under the assumption that $E[Y | X] = \beta^T X$ for some true unknown β (i.e. under the assumption that the true model is linear), this estimator is BLUE (best unbiased linear estimator), where “best” here means lowest variance.¹ This is the *Gauss-Markov Theorem*, and I am assuming that you saw it in Stat 346.

Here are a few pros and cons of linear regression to keep in mind. We will discuss these a lot, and also revisit them in our R demo.

- **Pro: efficiency:** If $n > p$, we have a closed-form solution. This means that the function is efficient to train. It is also really efficient to generate new predictions.
- **Con: identifiability:** If $n < p$, we cannot solve for $\hat{\beta}$ because there is no unique solution; the model is not identifiable.
- **Con: bias:** If the true model is not linear, we will have bias! Because the linear model might simply not be flexible enough to model the true relationship between the predictors and the response.
- **Pro/Con: variance:** Because it is not super flexible, linear regression should be a low-variance method. Unfortunately, when we start adding a lot of irrelevant or redundant predictors to the model, the variance can get very high.
- **Pro: interpretability:** Linear regression is interpretable! We can look at the estimated function $\hat{f}$ and say what variables are important, and in what direction they are contributing to our predictions.
- **Pro: usability:** Linear regression is easy to use “out-of-the-box” for a non-expert. There isn't anything to *tune* if we just regress y on all the variables (which we can do as long as $p < n$). Any refinement that the user wants to do after that is pretty easy to explain/understand.

3.2 KNN in high dimensions

We know that the best solution to (2) is $E[Y | X = x]$. So, we assume only that $E[Y | X = x]$ is smooth enough that:

$$E[Y | X = x] \approx E[Y | X \text{ is in a neighborhood of } x].$$

We then approximate this function directly by letting:

$$\hat{f}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i,$$

where $N_k(x)$ is the k training set datapoints that are closest to the desired test point x .

We didn't emphasize this point when we only had a single predictor variable, but the notion of “closest” datapoints requires a distance metric! Our distances are now measured in p -dimensional predictor space, and we actually have many different distance metrics that we could use. Unless otherwise specified, assume that we are using Euclidean

¹Do you remember what it means to call this a *linear* estimator? A hint is that it would still be linear even if we used polynomial features like X^2 .

distance. This means that:

$$N_1(x^{\text{test}}) = \arg \min_{i=1, \dots, n} \sqrt{\sum_{j=1}^p (x_j^{\text{test}} - x_{ij})^2} = \arg \min_{i=1, \dots, n} \|x^{\text{test}} - x_i\|_2.$$

Importantly, if your different predictors x are measured with different units, this neighbor function might not even make sense! This neighbor function treats all features the same. If one of our features is “price of house” and another feature is “number of bedrooms in house”, one of these has values in the hundreds of thousands and the other has values that are likely below 10. In this setting, the euclidean distance between points will be totally dominated by price, and bedrooms will be basically ignored. Because of this, you should almost certainly scale your features before applying KNN: usually we would do this by dividing every feature by its standard deviation. This creates a distance function where all features contribute equally. Below, we will talk about how this can lead to its own issues.

There are some pros and cons of KNN that are relevant regardless of the dimension p .

- **Con: usability:** we need to choose k . This means that KNN cannot be used directly “out of the box” - we probably want to choose k using cross-validation.
- **Pro/Con: bias/variance tradeoff:**
 - With small k , we can approximate functions that are arbitrarily wiggly. So we don’t need to worry that our function class $\mathcal{F}$ is not sufficiently flexible.
 - When k is small, each prediction is generated with very few datapoints, which means that we have high variance.
 - When k is large, a prediction might be generated with points that are actually far away from the test point x^{test} , which can introduce bias.
 - If we have enough data points n such that our predictor space is densely populated with training points, we can pick a pretty large k and still have it be the case that every training point in $N_k(x)$ is actually close to x . This means that we won’t have much bias, but we will also have low variance since k is large.
 - For a given problem, we may or may not find a “sweet-spot” k that makes KNN work really well!
- **Pro / Con: efficiency:**
 - Computational cost of training is essentially free. Just store the training dataset.
 - At testing, we need to compute distances between the new test point and all points in the training set. This could be slow if you implement it naively, but people have cool algorithms for finding nearest neighbors efficiently.

There are some additional cons of KNN that come up when the dimension p is large.

- **Impact of irrelevant features:** Suppose that we have p features in our dataset, but only a small subset of these features actually impact the response Y . All of these features are used in computing the neighbors of x ! So, we are letting totally irrelevant features contribute to our distance metric. This could introduce either bias or variance. The bias would be because I am doing a bad job selecting the meaningful neighbors. The variance would be because my prediction can vary based on extra random noise in some random irrelevant feature.
- **Lack of interpretability:** Unlike linear regression, running KNN on a high dimensional dataset tells you nothing at all about what features are most associated with Y . If you wanted to remove irrelevant features to improve performance, KNN provides no built in guidance for doing this. This is in contrast to linear regression.
- **Computational cost:** Storing the training set is actually quite expensive when n and p are large. So even though “training the model” just involves “storing the training set” - this is not free! Neither is computing lots of pairwise distances in high dimensions, or searching through high dimensional space for neighbors! We will need to come up with more clever algorithms for KNN to get around this.
- **Curse of dimensionality:** It turns out that, in high dimensions, points just become far from one another! So selecting neighbors in high dimensions is just a really hard task! The entire notion of neighbors hardly makes sense anymore, because our high-dimensional feature space will not be densely populated unless n is really large compared to p . So even though we can theoretically do KNN when $p > n$ (unlike for linear regression), it is going to work very poorly in this setting.

3.3 Comparing linear regression and KNN, without preprocessing

See the R Demo from class.

We ran out of time before we could talk about what to do about this! We will talk about that next time!

3.4 Miscellaneous

People asked good questions in class, so here are some extra points that I wanted to add!

- Another big advantage of linear regression over KNN is that we have theory that lets us do inference (e.g. prediction intervals!) to rigorously quantify our uncertainty for linear regression.
 - This inference depends on some number of assumptions about the underlying data generating mechanism. For example, the typical inference that we do assumes homoskedasticity (equal variance of our errors), but robust/sandwich standard errors let us relax this assumption (at the possible cost of statistical power).
 - We like that KNN requires no assumptions whatsoever: we don't need to believe that our model is truly linear, etc. But a downside of this lack of assumptions is that we don't have many tools available to make prediction intervals! This is a typical tradeoff: we would love to avoid assumptions, but usually avoiding assumptions also means we can't say much about our final answer!
 - Off the top of my head, I don't know much about inference for KNN. But I included some random thoughts below!
 - For KNN with $K = 3$, each prediction is based on three datapoints! Without additional assumptions, there is going to be a LOT of uncertainty associated with this prediction.
 - Later in the semester, we will talk about conformal inference, which lets us make prediction intervals for any machine learning model. This could be applied to KNN!
 - To make a prediction interval, we need both an estimate but also a measure of sampling uncertainty associated with this estimate. Maybe we could use bootstrapping to estimate the SE of a KNN prediction and make an interval based on that? When I googled "inference for KNN", I found a bunch of suggestions about bootstrapping! It would be interesting to try this out and see if it works!
 - * A sketch of the idea: Get B different KNN predictions for a point x^{test} by using different random bootstrap subsamples of the training dataset each time. Then, compute the standard deviation of these B predictions. Then, get your main prediction $\hat{y}^{\text{test}}$ using all of the data. Let $\hat{y}^{\text{test}} \pm 2SD(\hat{y}^{\text{test}})$ be your prediction interval. Would this cover the true y^{test} 95% of the time? I am not sure because KNN is strange and bootstrapping does have theory/requirements. Let's try it out in a future R demo during lecture.
- We mentioned that KNN requires storing the entire dataset to make test predictions, and that it is inefficient. This is not strictly speaking true, because people have made smart algorithms for getting around this problem. If you google "KNN with KD tree or Ball tree" you will find a lot of resources on fast algorithms. These are the ones that I learned about in my ML class, but I think a lot of others exist too!
- Someone asked if you need to store the entire training dataset to fit a linear regression model. It turns out that you don't! I will put a reading on your HW about "online learning" for linear regression. The idea is that, instead of storing our training dataset X, y all at once and computing $\hat{\beta} = (X^T X)^{-1} X^T y$, we can instead write $\hat{\beta}$ as:

$$\hat{\beta} = \arg \min_b \sum_{i=1}^n (y_i - x_i^T b)^2$$

and assume that we only have access to a single example x_i, y_i at a time. We have algorithms that can step along the gradient of this loss function according to one example x_i, y_i at a time in order to find the minimum. So the full data never needs to be stored!

- I'm sure there are things I forgot. Keep asking good questions!